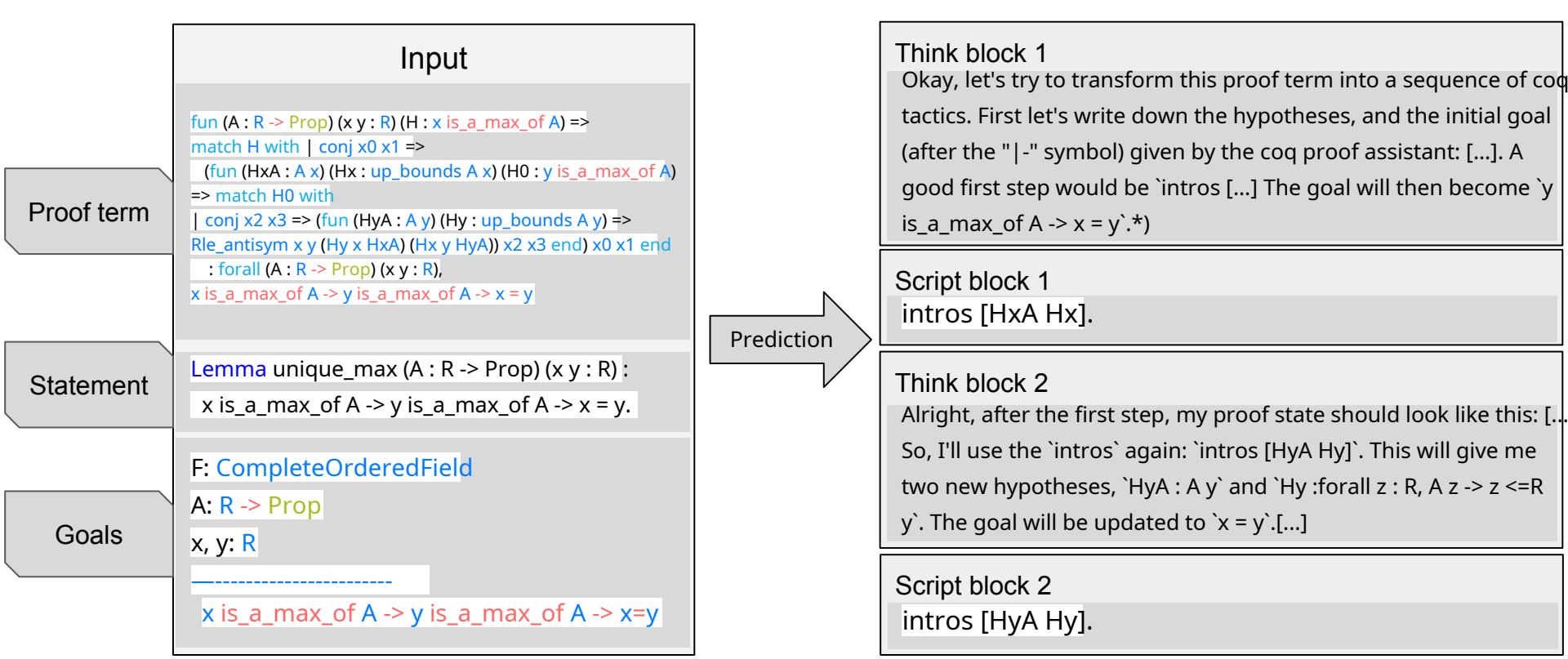
Input
Proof term
fun (A : R -> Prop) (x y : R) (H : x is_a_max_of A) =>
match H with | conj x0 x1 =>
(fun (HxA : A x) (Hx : up_bounds A x) (H0 : y is_a_max_of A)
=> match H0 with
| conj x2 x3 => (fun (HyA : A y) (Hy : up_bounds A y) =>
Rle_antisym x y (Hy x HxA) (Hx y HyA)) x2 x3 end) x0 x1 end
: forall (A : R -> Prop) (x y : R),
x is_a_max_of A -> y is_a_max_of A -> x = y
Statement
Lemma unique_max (A : R -> Prop) (x y : R) :
x is_a_max_of A -> y is_a_max_of A -> x = y.
Goals
F: CompleteOrderedField
A: R -> Prop
x, y: R
——————————————
x is_a_max_of A -> y is_a_max_of A -> x=y
Prediction
Think block 1
Okay, let's try to transform this proof term into a sequence of coq tactics. First let's write down the hypotheses, and the initial goal (after the "|-" symbol) given by the coq proof assistant: [...]. A good first step would be `intros [...] The goal will then become `y is_a_max_of A -> x = y`.*)
Script block 1
intros [HxA Hx].
Think block 2
Alright, after the first step, my proof state should look like this: [...] So, I'll use the `intros` again: `intros [HyA Hy]`. This will give me two new hypotheses, `HyA : A y` and `Hy :forall z : R, A z -> z <=R y`. The goal will be updated to `x = y`.[...]
Script block 2
intros [HyA Hy].